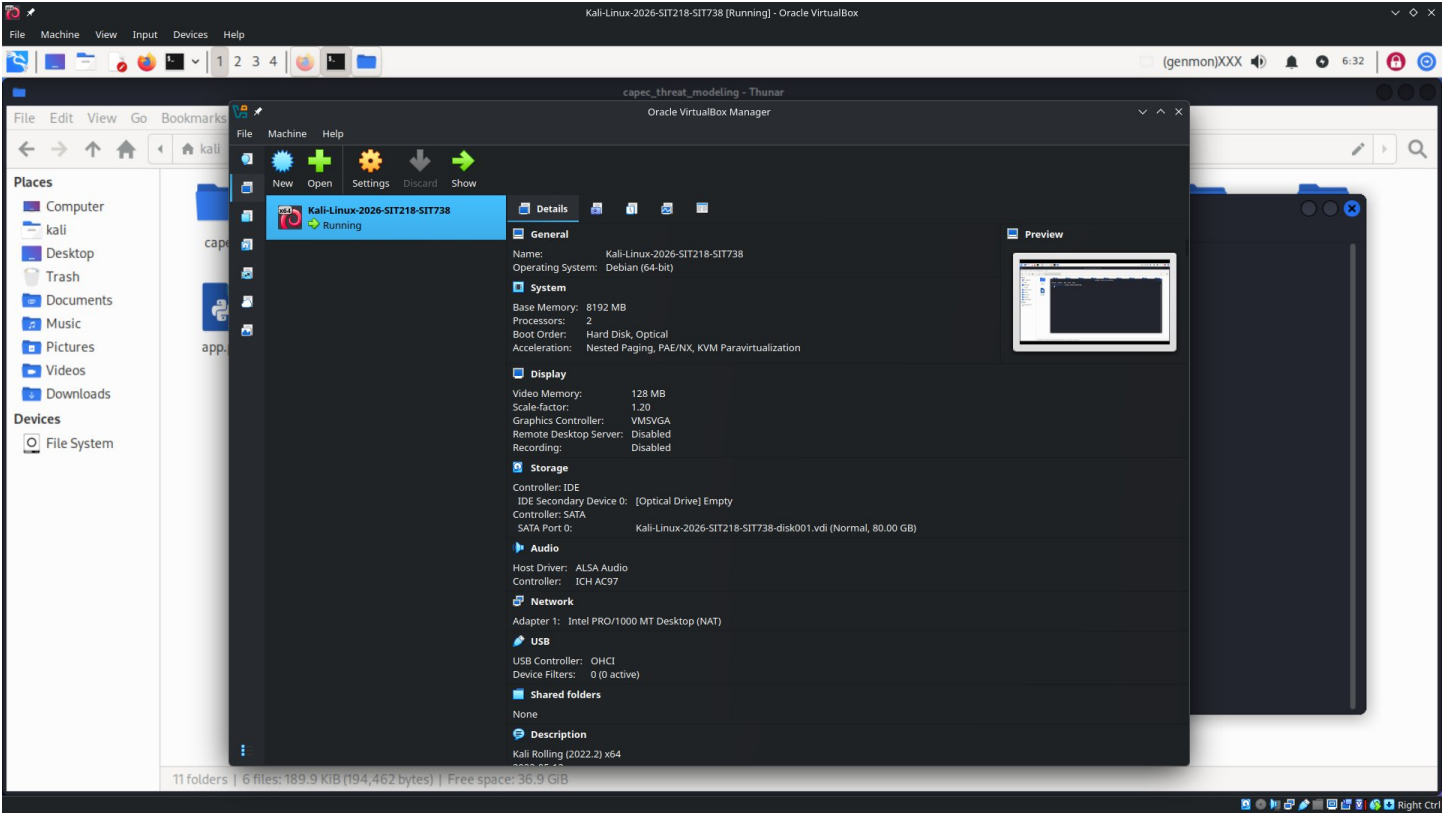


Task 2.2P

I first set up the virtual machine.



I then started the web server:

```
(kali@kali)-[~/capec_threat_modeling]
$ export TRANSFORMERS_OFFLINE=1
export HF_DATASETS_OFFLINE=1
export HF_HUB_OFFLINE=1

(kali@kali)-[~/capec_threat_modeling]
$ source venv/bin/activate

(venv)-(kali@kali)-[~/capec_threat_modeling]
$ python3 app.py
Loading weights: 100% | 103/103 [00:00<00:00, 18330.50it/s]
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.2.15:5000
Press CTRL+C to quit
* Restarting with stat
Loading weights: 100% | 103/103 [00:00<00:00, 13494.51it/s]
* Debugger is active!
* Debugger PIN: 142-444-250
```

Then, I copied the initial description from Task 2.1P's task sheet and pressed decompose system, and followed the rest of the instructions.

← → ↻ 🏠

🔍 http://127.0.0.1:5000

☆

👤 📄 ☰

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

LLM-Based CAPEC Threat Modeling Tool

System Description:

The web application is portal for performing static code analysis of software. The users of the application are software developers (who want to analyse their code), their managers, and administrators (of the web application). The users are authenticated using a cloud based active directory server which manages their identities. The software developers can upload their code to the web service and perform a static code analysis (which is run in the application server) on the portal, view the analysis reports (scan results stored in the database) and prioritise the vulnerabilities to be addressed and resolve the fixed vulnerabilities (stored in the database). The software developers can also use their own IDEs to interact with code analyser via the web services API and perform the static code analysis. Their managers can view the vulnerabilities reported by the scan tool, view the progress made by the development team in fixing the vulnerabilities. The administrators can manage the whole application.

Decompose system

Kali Linux-2026-SIT218-SIT738 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

🔍 (genmon)XXX 6:38

LLM-Based CAPEC Threat Modeling Tool — Mozilla Firefox

LLM-Based CAPEC Threat Modeling Tool x +

← → ↻ 🏠

🔍 http://127.0.0.1:5000

80% ☆

👤 📄 ☰

LLM-Based CAPEC Threat Modeling Tool — Mozilla Firefox

Decompose system

Stage 1: System Decomposition

External Entities:

- User
- Administrator
- Software Developer
- Manager
- Active Directory Server

Processes:

- Authentication Service
- Payment Processing
- User Management (Active Directory)
- Static Code Analysis Tool
- Web Server
- Database
- IDE Integration (via API)

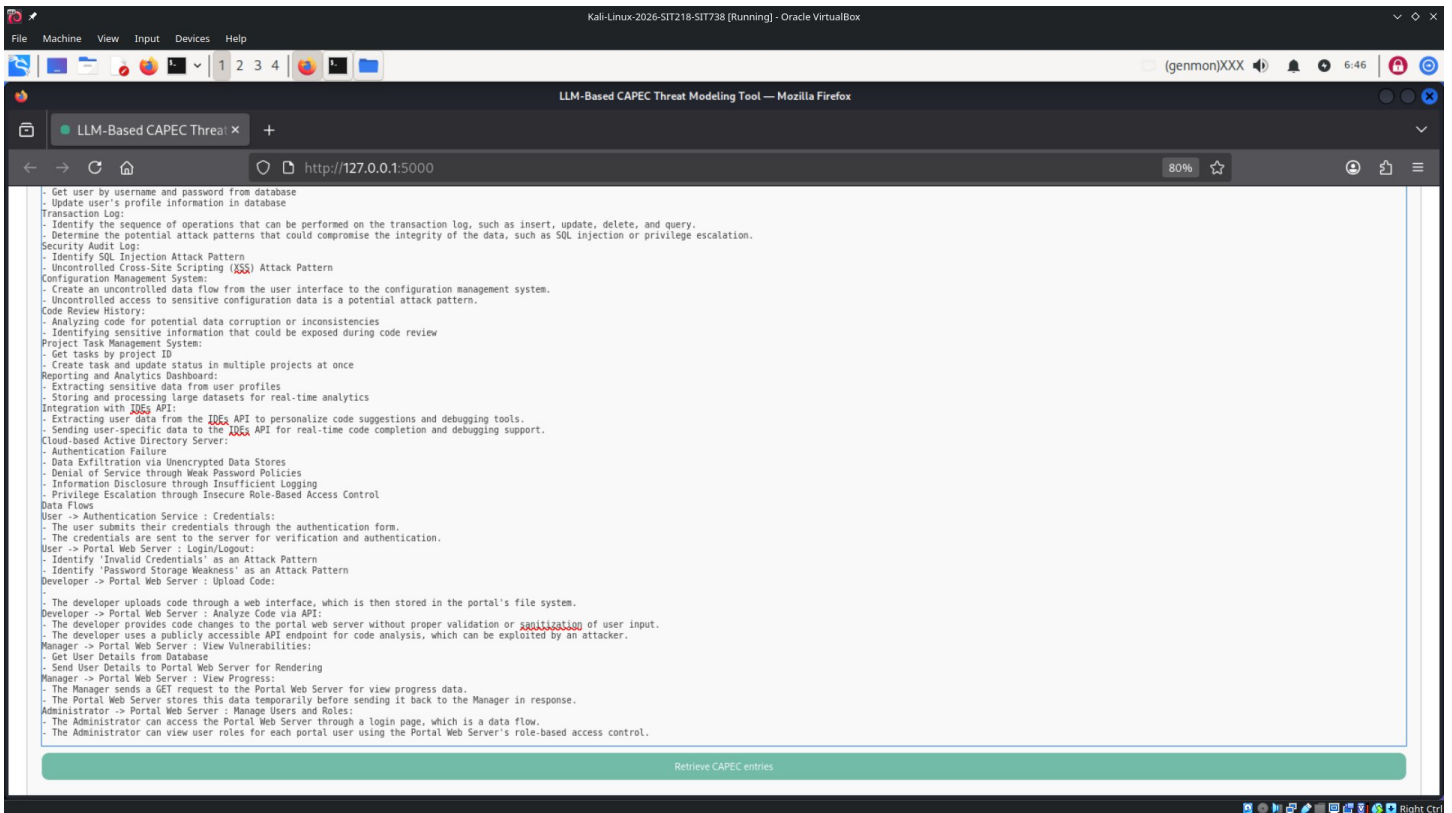
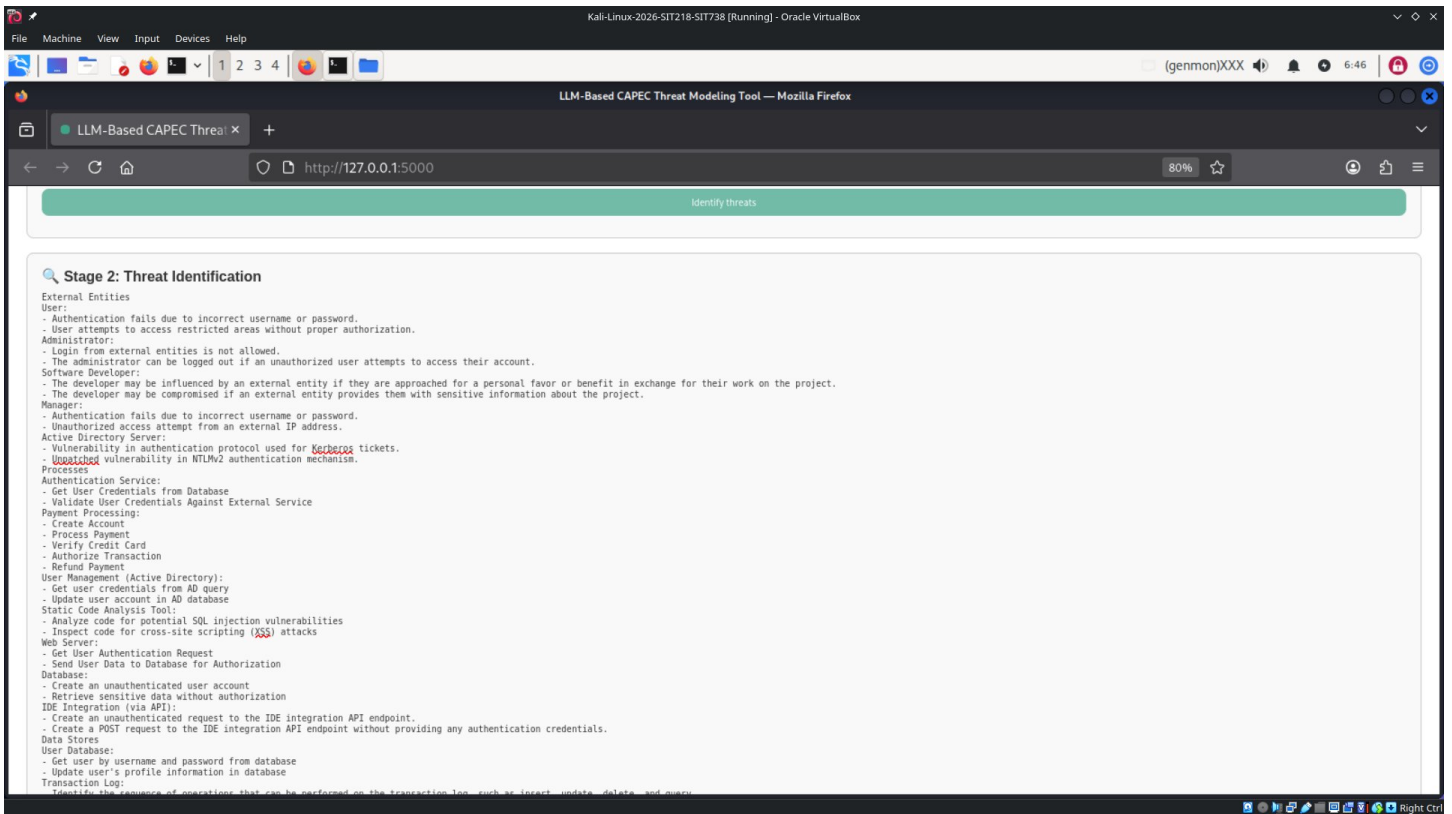
Data Stores:

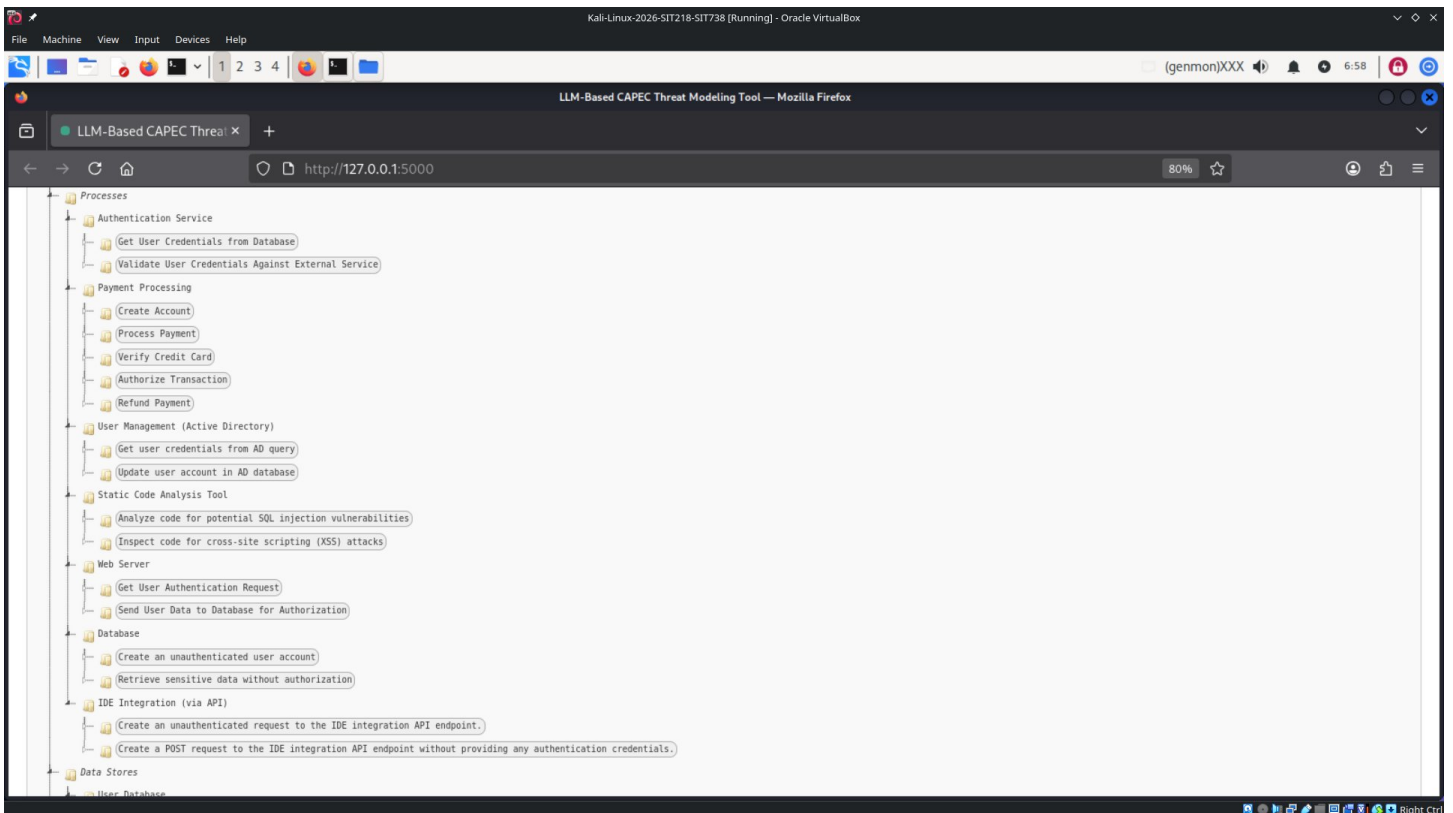
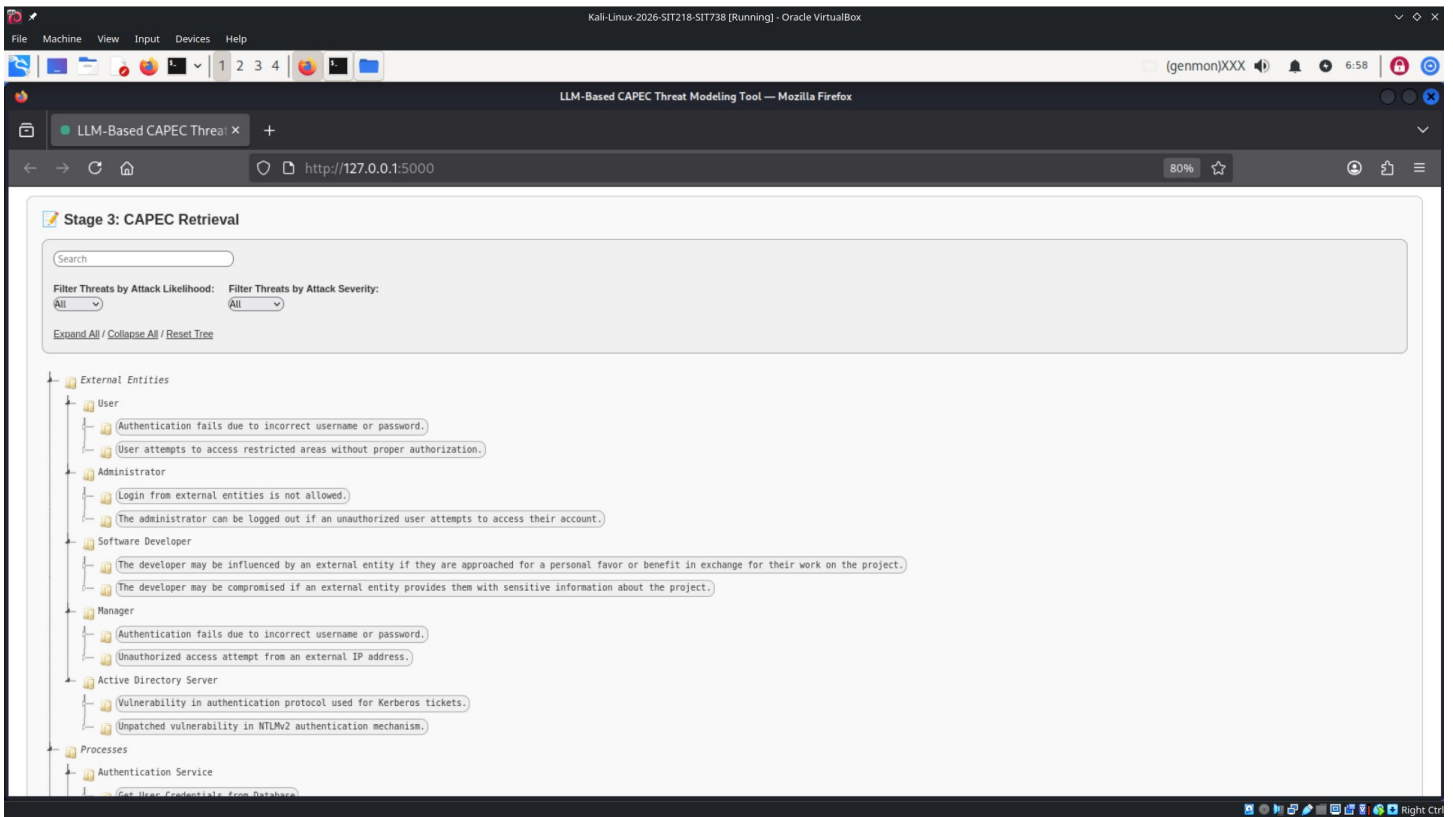
- User Database
- Transaction Log
- Security Audit Log
- Configuration Management System
- Code Review History
- Project Task Management System
- Reporting and Analytics Dashboard
- Integration with IDEs API
- Cloud-based Active Directory Server

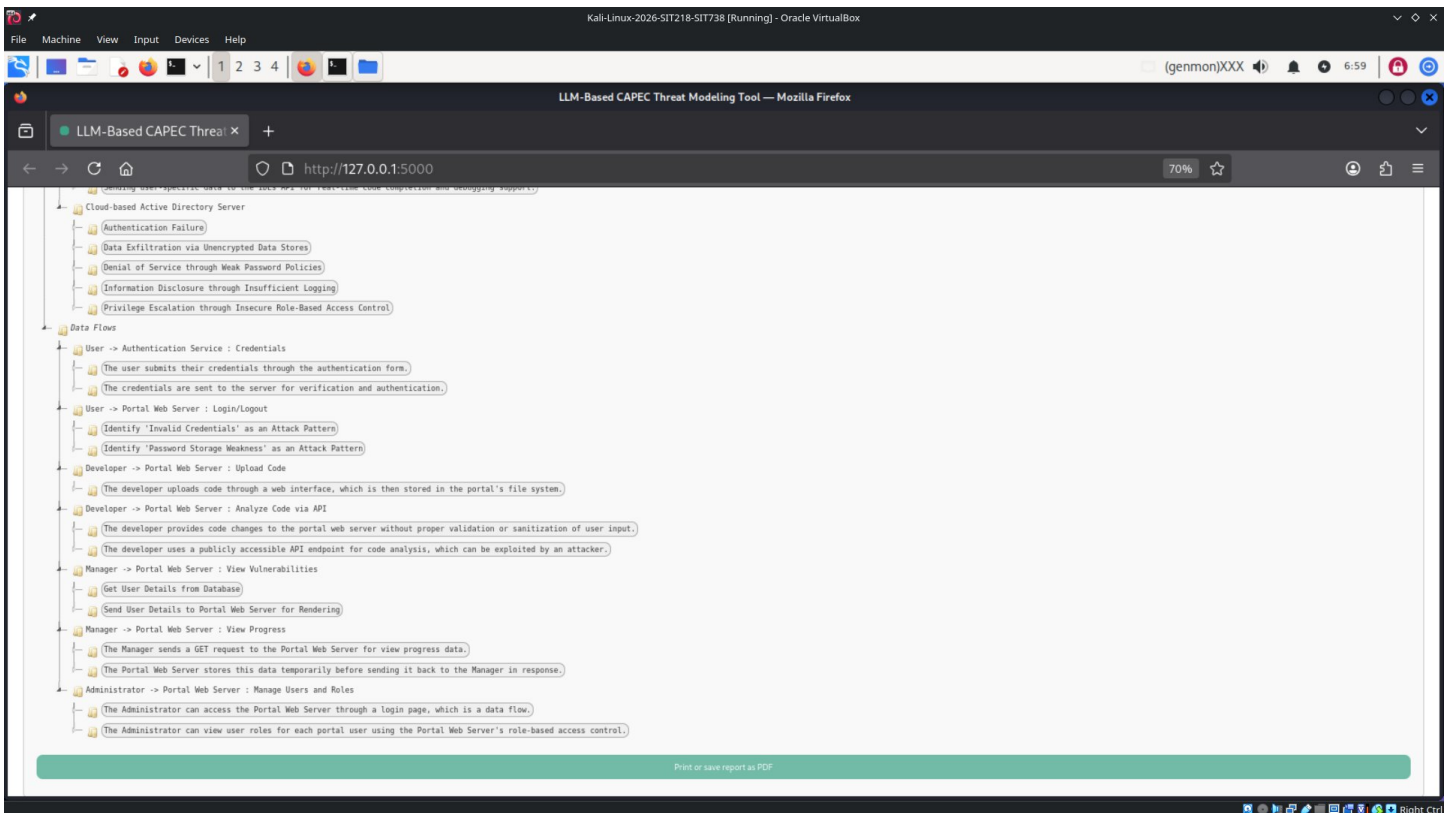
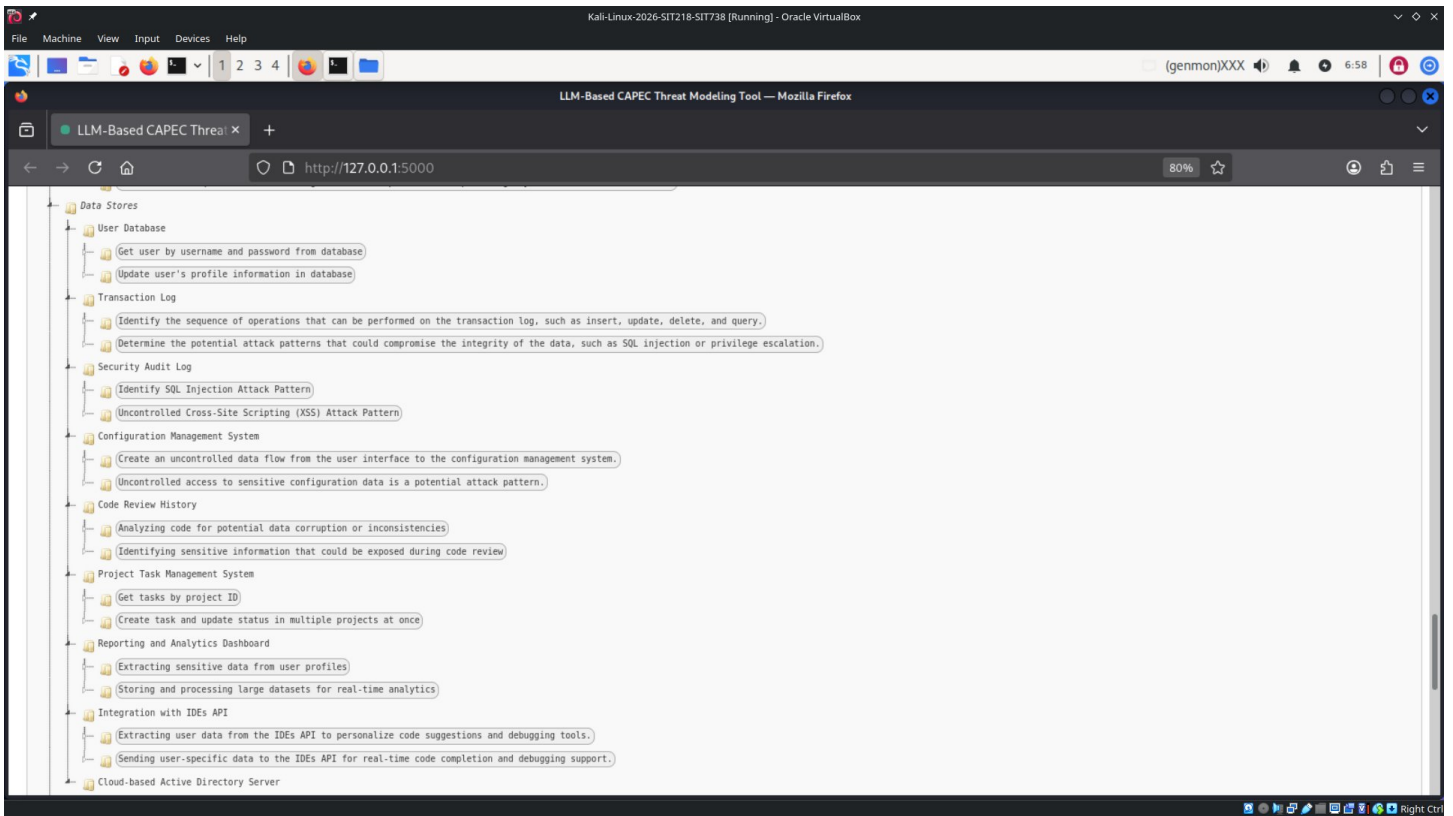
Data Flows:

- User -> Authentication Service : Credentials
- User -> Portal Web Server : Login/Logout
- Developer -> Portal Web Server : Upload Code
- Developer -> Portal Web Server : Analyze Code via API
- Manager -> Portal Web Server : View Vulnerabilities
- Administrator -> Portal Web Server : View Progress
- Administrator -> Portal Web Server : Manage Users and Roles

Identify threats







```
(venv)kali@kali: ~/capec_threat_modeling
Session Actions Edit View Help
d Payment\nUser Management (Active Directory):\n- Get user credentials from AD query\n- Update user account in AD database\nStatic Code Analysis
Tool:\n- Analyze code for potential SQL injection vulnerabilities\n- Inspect code for cross-site scripting (XSS) attacks\nWeb Server:\n- Get Us
er Authentication Request\n- Send User Data to Database for Authorization\nDatabase:\n- Create an unauthenticated user account\n- Retrieve sensi
tive data without authorization\nIDE Integration (via API):\n- Create an unauthenticated request to the IDE integration API endpoint.\n- Create
a POST request to the IDE integration API endpoint without providing any authentication credentials.\nData Stores\nUser Database:\n- Get user by
username and password from database\n- Update user's profile information in database\nTransaction Log:\n- Identify the sequence of operations t
hat can be performed on the transaction log, such as insert, update, delete, and query.\n- Determine the potential attack patterns that could co
mpromise the integrity of the data, such as SQL injection or privilege escalation.\nSecurity Audit Log:\n- Identify SQL Injection Attack Pattern
\n- Uncontrolled Cross-Site Scripting (XSS) Attack Pattern\nConfiguration Management System:\n- Create an uncontrolled data flow from the user i
nterface to the configuration management system.\n- Uncontrolled access to sensitive configuration data is a potential attack pattern.\nCode Rev
iew History:\n- Analyzing code for potential data corruption or inconsistencies\n- Identifying sensitive information that could be exposed durin
g code review\nProject Task Management System:\n- Get tasks by project ID\n- Create task and update status in multiple projects at once\nReporti
ng and Analytics Dashboard:\n- Extracting sensitive data from user profiles\n- Storing and processing large datasets for real-time analytics\nIn
tegration with IDEs API:\n- Extracting user data from the IDEs API to personalize code suggestions and debugging tools.\n- Sending user-specific
data to the IDEs API for real-time code completion and debugging support.\nCloud-based Active Directory Server:\n- Authentication Failure\n- Da
ta Exfiltration via Unencrypted Data Stores\n- Denial of Service through Weak Password Policies\n- Information Disclosure through Insufficient L
ogging\n- Privilege Escalation through Insecure Role-Based Access Control\nData Flows\nUser → Authentication Service : Credentials:\n- The user
submits their credentials through the authentication form.\n- The credentials are sent to the server for verification and authentication.\nUser
→ Portal Web Server : Login/Logout:\n- Identify 'Invalid Credentials' as an Attack Pattern\n- Identify 'Password Storage Weakness' as an Attac
k Pattern\nDeveloper → Portal Web Server : Upload Code:\n- The developer uploads code through a web interface, which is then stored in the p
ortal's file system.\nDeveloper → Portal Web Server : Analyze Code via API:\n- The developer provides code changes to the portal web server wit
hout proper validation or sanitization of user input.\n- The developer uses a publicly accessible API endpoint for code analysis, which can be e
xploited by an attacker.\nManager → Portal Web Server : View Vulnerabilities:\n- Get User Details from Database\n- Send User Details to Portal
Web Server for Rendering\nManager → Portal Web Server : View Progress:\n- The Manager sends a GET request to the Portal Web Server for view pro
gress data.\n- The Portal Web Server stores this data temporarily before sending it back to the Manager in response.\nAdministrator → Portal We
b Server : Manage Users and Roles:\n- The Administrator can access the Portal Web Server through a login page, which is a data flow.\n- The Admi
nistrator can view user roles for each portal user using the Portal Web Server's role-based access control."

127.0.0.1 - - [22/Jul/2026 06:56:54] "POST /retrieve_entries HTTP/1.1" 200 -
```

Once you are done playing around, you can press CTRL+C to exit out of the flask web server and then run the `deactivate` command to exit out of the activated python virtual environment named venv.

```
127.0.0.1 - - [22/Jul/2026 06:56:54] "POST /retrieve_entries HTTP/1.1" 200 -
^C^C
(venv)-(kali@kali)-[~/capec_threat_modeling]
$ deactivate
(kali@kali)-[~/capec_threat_modeling]
$
```

Video Demonstration: <https://youtu.be/Qb3PtWuJxrw>

LLM-Based CAPEC Threat Modeling Tool

System Description:

The web application is portal for performing static code analysis of software. The users of the application are software developers (who want to analyse their code), their managers, and administrators (of the web application). The users are authenticated using a cloud based active directory server which manages their identities. The software developers can upload their code to the webserver and perform a static code analysis (which is run in the application server) on the portal view the analysis reports (scan results stored in the database) and

Stage 1: System Decomposition

External Entities:

- User
- Administrator
- Software Developer
- Manager
- Active Directory Server

Processes:

- Authentication Service
- Payment Processing
- User Management (Active Directory)
- Static Code Analysis Tool
- Web Server
- Database
- IDE Integration (via API)

Data Stores:

- User Database
- Transaction Log
- Security Audit Log
- Configuration Management System
- Code Review History
- Project Task Management System
- Reporting and Analytics Dashboard
- Integration with IDEs API
- Cloud-based Active Directory Server

Data Flows:

- User -> Authentication Service : Credentials
- User -> Portal Web Server : Login/Logout
- Developer -> Portal Web Server : Upload Code
- Developer -> Portal Web Server : Analyze Code via API
- Manager -> Portal Web Server : View Vulnerabilities
- Manager -> Portal Web Server : View Progress
- Administrator -> Portal Web Server : Manage Users and Roles

Stage 2: Threat Identification

External Entities

User:

- Authentication fails due to incorrect username or password.
- User attempts to access restricted areas without proper authorization.

Administrator:

- Login from external entities is not allowed.
- The administrator can be logged out if an unauthorized user attempts to access their account.

Software Developer:

- The developer may be influenced by an external entity if they are approached for a personal favor or benefit in exchange for their work on the project.
- The developer may be compromised if an external entity provides them with sensitive information about the project.

Manager:

- Authentication fails due to incorrect username or password.
- Unauthorized access attempt from an external IP address.

Active Directory Server:

- Vulnerability in authentication protocol used for Kerberos tickets.
- Unpatched vulnerability in NTLMv2 authentication mechanism.

Processes

Authentication Service:

- Get User Credentials from Database
- Validate User Credentials Against External Service

Payment Processing:

- Create Account
- Process Payment
- Verify Credit Card
- Authorize Transaction
- Refund Payment

User Management (Active Directory):

- Get user credentials from AD query
- Update user account in AD database

Static Code Analysis Tool:

- Analyze code for potential SQL injection vulnerabilities
- Inspect code for cross-site scripting (XSS) attacks

Web Server:

- Get User Authentication Request
- Send User Data to Database for Authorization

Database:

- Create an unauthenticated user account
- Retrieve sensitive data without authorization

IDE Integration (via API):

- Create an unauthenticated request to the IDE integration API endpoint.
- Create a POST request to the IDE integration API endpoint without providing any authentication credentials.

Data Stores

User Database:

- Get user by username and password from database
- Update user's profile information in database

Transaction Log:

- Identify the sequence of operations that can be performed on the transaction log, such as insert, update, delete, and query.
- Determine the potential attack patterns that could compromise the integrity of the data, such as SQL injection or privilege escalation.

Security Audit Log:

- Identify SQL Injection Attack Pattern
- Uncontrolled Cross-Site Scripting (XSS) Attack Pattern

Configuration Management System:

- Create an uncontrolled data flow from the user interface to the configuration management system.
- Uncontrolled access to sensitive configuration data is a potential attack pattern.

Code Review History:

- Analyzing code for potential data corruption or inconsistencies
- Identifying sensitive information that could be exposed during code review

Project Task Management System:

- Get tasks by project ID
- Create task and update status in multiple projects at once

Reporting and Analytics Dashboard:

- Extracting sensitive data from user profiles
- Storing and processing large datasets for real-time analytics

Integration with IDEs API:

- Extracting user data from the IDEs API to personalize code suggestions and debugging tools.
- Sending user-specific data to the IDEs API for real-time code completion and debugging support.

Cloud-based Active Directory Server:

- Authentication Failure
- Data Exfiltration via Unencrypted Data Stores
- Denial of Service through Weak Password Policies
- Information Disclosure through Insufficient Logging
- Privilege Escalation through Insecure Role-Based Access Control

Data Flows**User -> Authentication Service : Credentials:**

- The user submits their credentials through the authentication form.
- The credentials are sent to the server for verification and authentication.

User -> Portal Web Server : Login/Logout:

- Identify 'Invalid Credentials' as an Attack Pattern
- Identify 'Password Storage Weakness' as an Attack Pattern

Developer -> Portal Web Server : Upload Code:

-
- The developer uploads code through a web interface, which is then stored in the portal's file system.

Developer -> Portal Web Server : Analyze Code via API:

- The developer provides code changes to the portal web server without proper validation or sanitization of user input.
- The developer uses a publicly accessible API endpoint for code analysis, which can be exploited by an attacker.

Manager -> Portal Web Server : View Vulnerabilities:

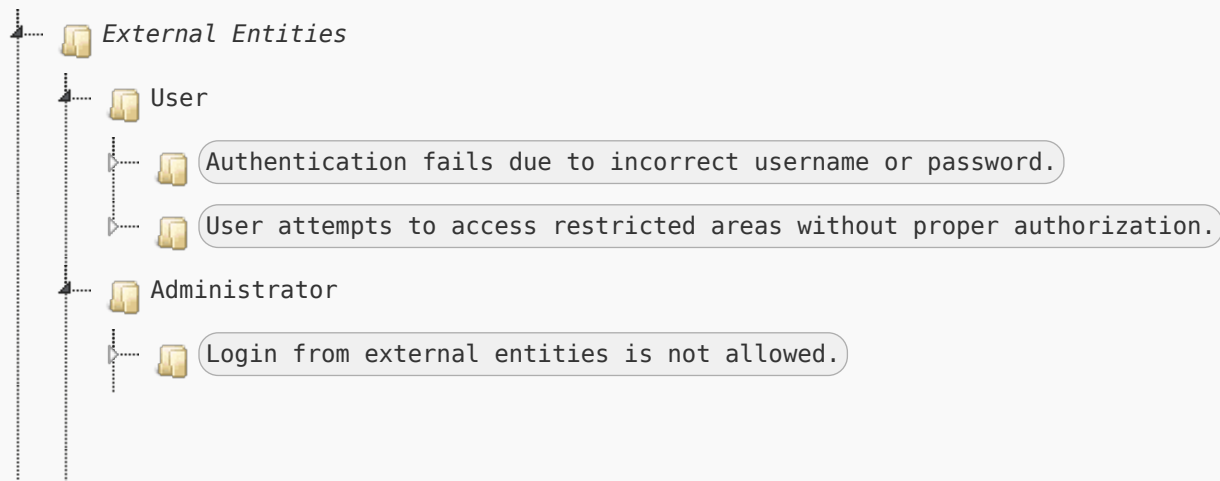
- Get User Details from Database
- Send User Details to Portal Web Server for Rendering

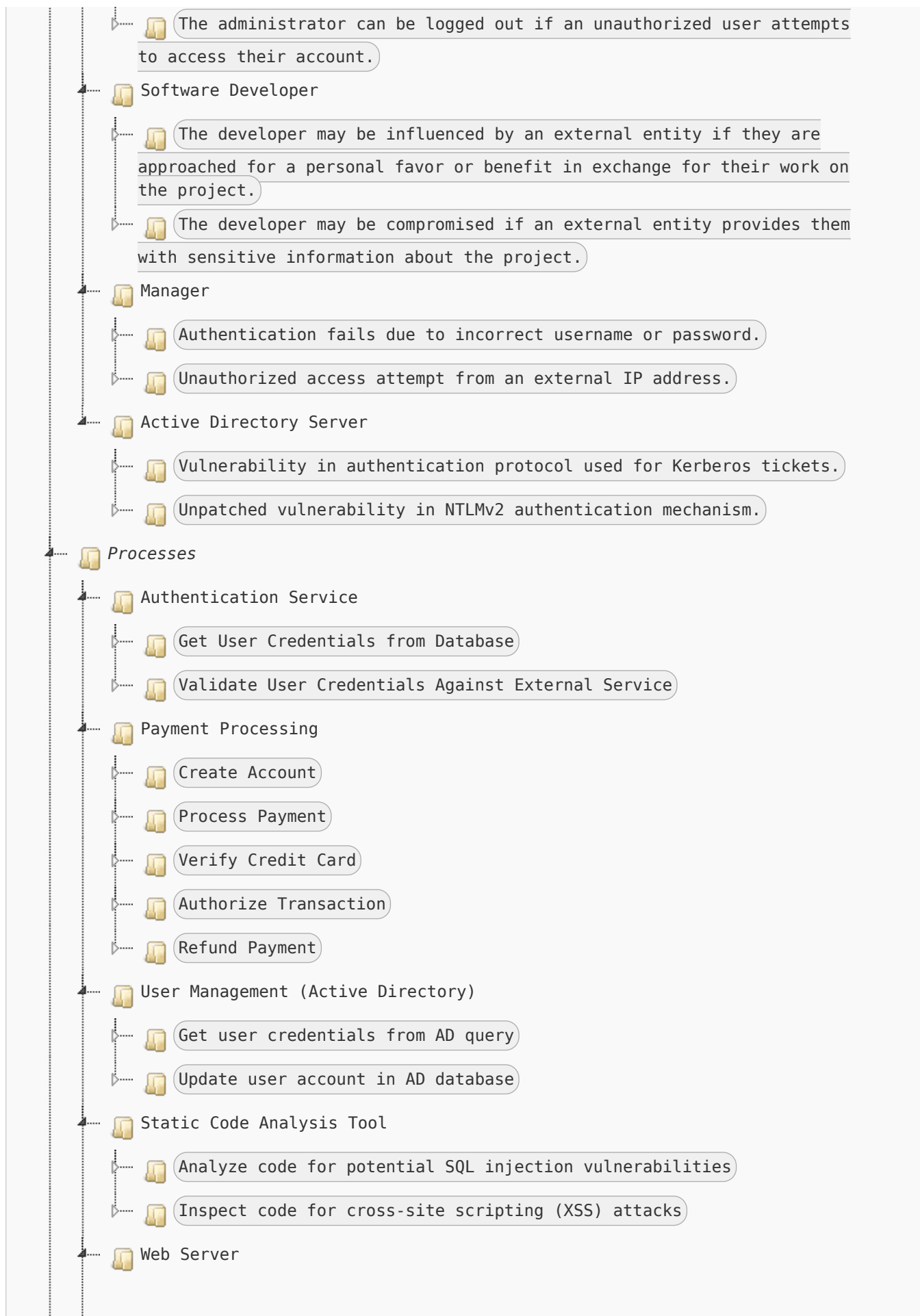
Manager -> Portal Web Server : View Progress:

- The Manager sends a GET request to the Portal Web Server for view progress data.
- The Portal Web Server stores this data temporarily before sending it back to the Manager in response.

Administrator -> Portal Web Server : Manage Users and Roles:

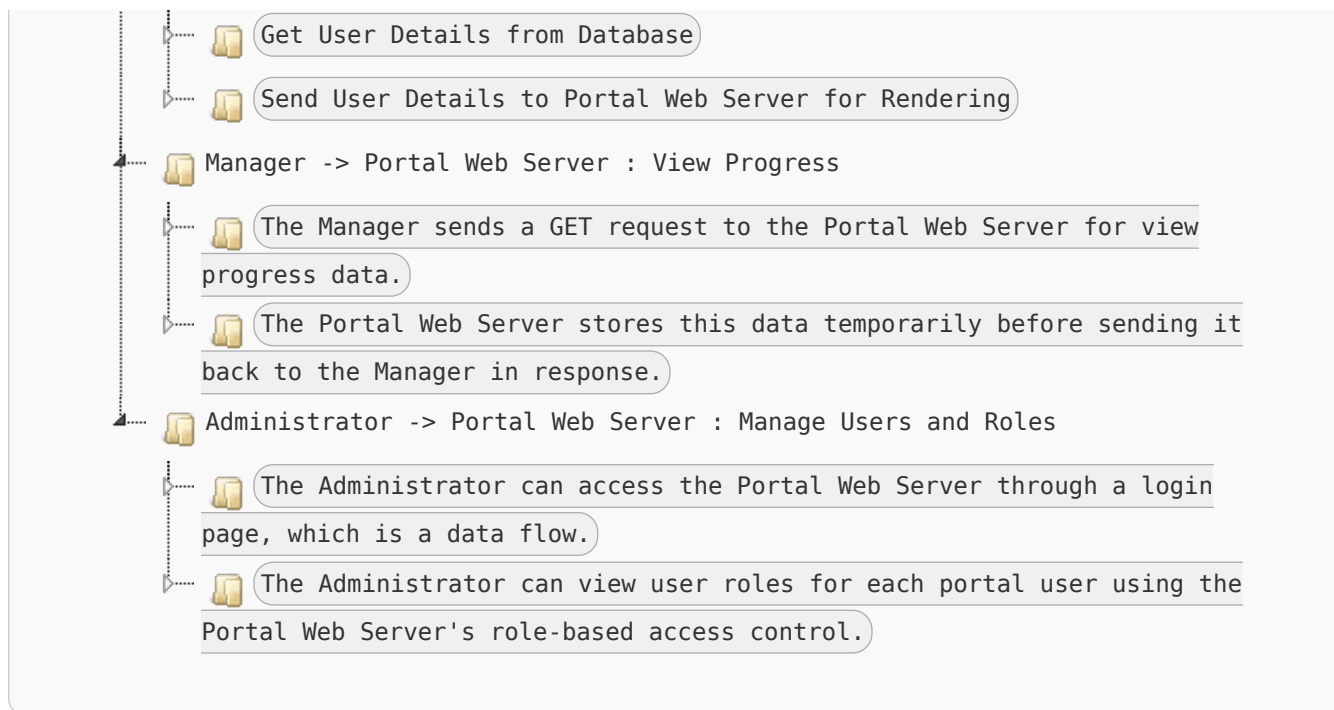
- The Administrator can access the Portal Web Server through a login page, which is a data flow.
- The Administrator can view user roles for each portal user using the Portal Web Server's role-based access control.

Stage 3: CAPEC Retrieval









© 2025 Karim Sammouri. Licensed under MIT.